CSE-215 Lab Report

Project: Online Quiz App

Faculty: Dr. Mohammad Shifat-E-Rabbi (MSRb)

Lab Instructor: Rejuana Islam

Group: 01	
MD. Yasir Arafat	2412558042
Alvee Ifteshum	2411387642
Syed Amin Hossain	2414265042

Abstract

In this project, We have implemented Object-oriented programming by using Java Programming Language. We have used all basic functionality of Object-oriented programming like inheritance, polymorphism etc. The program also utilizes file I/O in order to store and retrieve data from local drives giving the user access to the contents locally. The whole program has been done

dynamically. Therefore, this app adapts to different situations.

Introduction

"QuizApp" is based on a multi-user application. Both Teacher and Student have different sections for them. Also there is a leaderboard for students to understand their postion. As soon as, teacher creates a new course, students can see that on their enrolled option. If the teacher adds a new quiz on a certain course, the student can just go back to that course and take the quiz. Also, there is a login and signUp system for teachers and students. To run the quizApp, you just need to run the App.java file.

Methodology

The program tree is structured as below:

```
>main/
    >java/
        >com.quizapp/
            >Actions/
                ActionClasses
            >Controllers/
                ControllersClasses
            App.java
    >resources/
        >com.quizapp/
            >fxml files
        Other resources files
```

App.java: This heart of the whole project. To run the app, we have to simply run this file. This is the only file that contains the main class. By running this file app will start with the login page.

resources/com.quizapp: Here are the javaFx files in fxml format. Fxml works for static pages but it also gives a gateway to dynamic pages by using a controller. For every page on this application there is a fxml file in this folder.

Controllers: Controllers classes initialize the corresponding fxml file. controllers have been used to create dynamic pages for the app. In another word, fxml gives the structure and the controller uses it.

Actions: Actions are mostly the **parent** class of the controllers in this app. Action classes contain the major logic of that page. Making Controllers free to work with Javafx.

Working Mechanism

App starts with the login page. If they don't have an account they can go to the SignUp page.

After login, students can see their courses taken on the home page. Also they can see "take quiz" option for every course. If they select the "take quiz" option then, they will be shifted to StudentQuizList where they can choose which quiz they want to choose. On every quiz they took their leaderboard score increases.

If a teacher login, they will see their current course. They are given the option to edit the course and add a course.

Login System

The login system of the program is structured with the following parameters

- Username
- Password

When a user enters the username or password it checks for information from the credentials folder. Teacher.csv and student.csv.

SignUp System

Both teacher and student can signUp to the app very easily from a fully guided page.

Dynamic Pages: This App contains a lot of dynamic pages. On dynamic pages contents are loaded on demand. For example, if there are two courses offered by the teachers, students will see two courses on the page. If teachers are offering ten courses, they see ten courses on the page. Many of the pages are made dynamic due the nature of the project. Some of the dynamic pages:

- Teacher Main Page
- Student Main Page
- Enrolled Page
- Take quiz page
- Leaderboard Page

Findings

- Working with files is pretty hard. Therefore a database would be a better option.
- Writing the whole JavaFx code in one page is almost impossible. That's why dividing a page in different files is a good option.
- With JavaFx we can build professional apps.
- Intensive testing needed to fix all the bugs
- OOP code is organized into classes and objects, which are logical units representing real-world entities. This modularity makes it easier to understand and modify individual parts of the code without affecting the entire system

Plans

- Revise the code to fix bugs and come out with better design.
- Make the Sign in interface better and introduce it more functionally such as Sign up.
- Add a Recovery Key system, in case the user forgot his/her password.
- Add a sign out option
- Contact all the page all together
- Test the project with better polls

- Make some details analyzing page where teacher and student can get better feedback
- Use dynamic memory allocation by using free and malloc. It will give better stability to the program.

Conclusion

"QuizApp" is a straightforward project that leverages OOP concepts. The program is modular and has the potential for further expansion with additional features and functionalities. Working on this project provided valuable experiences, including numerous learning opportunities, bug fixing, and UI implementation.